EE G7601 · FINAL PROJECT

Phishing Email Detection

Utilizing an Agentic AI Approach

Gabriel Olufade · Seongjae Shin · Imad Ali Syed

Department of Cybersecurity

CEAS_08 Corpus | 39,154 Emails | Four Cooperating Agents

AGENDA

Presentation Outline

01	**The Problem**	Why phishing detection still matters
02	**Related Work**	Prior approaches and how we differ
03	**Dataset**	Multi-source datasets and external evaluation
04	**Agentic Architecture**	Four cooperating agents
05	**Agent Designs**	Preprocessing → Keywords → NLP → Reasoning
06	**Evaluation & Results**	Metrics, confusion matrix, interpretability
07	**Discussion & Future Work**	Limitations and next steps

Phishing: a Persistent and Evolving Threat

Phishing emails remain one of the most prevalent and effective vectors of cyber-attack.

Attackers craft deceptive emails that mimic trusted senders and pressure users into revealing credentials, downloading malware, or transferring funds. As organizations grow and communication channels multiply, distinguishing legitimate mail from a sophisticated spoof becomes increasingly difficult for the end user.

An automated, accurate, and interpretable detector is therefore essential for protecting personal data, credentials, and organizational assets.

Why this matters

User confusion

Spoofed emails closely resemble legitimate ones, especially across mobile clients.

Cost of a missed phish

Every undetected phishing email is a potential credential or data compromise.

Drift over time

Attacker language evolves; detectors must be modular and inspectable.

PROJECT APPROACH

Goal: an Interpretable, Modular Detector

Decompose phishing detection into four cooperating agents — each with a single, well-scoped role.

Interpretability

Each agent emits structured signals (keyword scores, URL counts, classifier probabilities) that can be inspected directly.

Modularity

Any agent can be replaced or upgraded — e.g., swap TF-IDF for a transformer — without re-engineering the rest of the pipeline.

Reproducibility

Built only with widely available Python libraries (pandas, NLTK, scikit-learn) and a fixed random seed.

BACKGROUND

Related Work

Reference	Approach	Relevance to Our Work
Pimpason et al. (2025)	Deep learning + NLP (BERT, CNN, RNN) for phishing detection.	*We split the task into specialized agents instead of one monolithic model.*
Anilkumar et al. (2023)	Survey of NLP + ML approaches and feature extraction techniques.	*Informed our text-feature choices; we add an explicit reasoning layer.*
Sahingoz et al. (2019)	Logistic Regression, Random Forest, and SVM over textual features.	*Validates effectiveness of text features; our pipeline adds rule-based agents.*
Helmer et al. (2002)	Multi-agent intrusion detection with cleaner / low-/high-level agents.	*Inspired our agent decomposition and inter-agent contracts.*
Korkmaz (2026)	TF-IDF + ML and DL benchmarks; standard evaluation metrics.	*Drove our choice of TF-IDF + Logistic Regression and metric set.*

DATA

Dataset: CEAS_08

39,154

Total labeled emails in the corpus

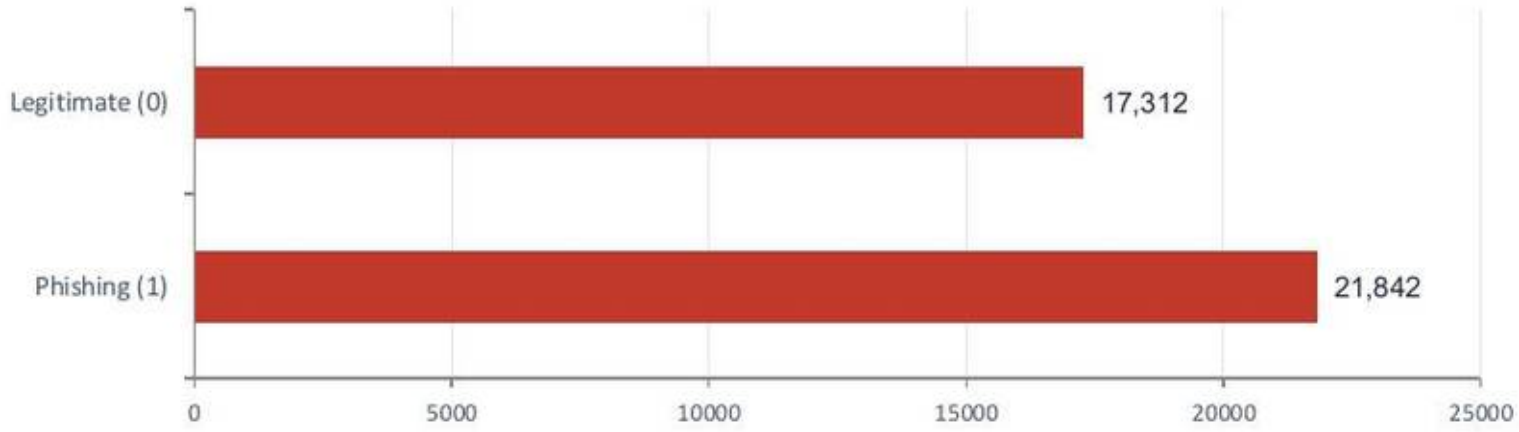
21,842

Phishing emails (≈55.8%)

17,312

Legitimate emails (≈44.2%)

Class Balance



Splits and seed

Stratified 80 / 20 train–test split (random_state = 42)

Held-out test set: **7,831 emails**

Identical split reused by Agent 3 and Agent 4 to avoid leakage

PIPELINE OVERVIEW

System Architecture

Four cooperating agents pass a shared pandas DataFrame; each appends new columns to it.



Data flow

Raw email → cleaned text + URL meta → keyword score + risk → TF-IDF probability → **binary verdict**

AGENT DESIGN

Agent 1 — Email Preprocessing

Responsibilities

- Load CEAS_08 and drop unused columns
- Lowercase text; strip HTML, URLs, digits, punctuation
- Concatenate clean subject + body into a canonical text field
- Tokenize with NLTK word_tokenize
- Preserve URL counts (has_url, url_count) before stripping

Output contract

DataFrame with clean_subject_body, CSB_tokens, has_url, and url_count columns — the uniform input every downstream agent depends on.

Cleaning Pipeline

1	Load CSV → drop sender / receiver / date / urls
2	Lowercase · strip <html> · remove URLs
3	Remove digits & punctuation · collapse whitespace
4	Build clean_subject_body
5	Tokenize → CSB_tokens
6	Compute url_count and has_url

AGENT DESIGN

Agent 2 — Keyword & Pattern Detection

Rule-based phishing score

Two weighted dictionaries

Suspicious words (urgent, password, login, suspended, bank...) + multi-word regex phrases ("verify your account", "reset your password", "click here").

Stem-aware matching

Porter stemmer normalizes morphological variants — "verifying" / "verified" still match the verify entry.

Caps and a co-occurrence boost

Per-token caps (3 keywords / 2 phrases) prevent any one term from dominating; ×1.15 boost when both signal types fire together.

URL evidence folded in

+1.0 for one URL, +2.0 more when ≥2 URLs are present.

Risk Buckets

LOW	score < 4.0
MEDIUM	4.0 ≤ score < 8.0
HIGH	score ≥ 8.0

Outputs to DataFrame

- **agent2_score** · numeric phishing score
- **agent2_risk** · low / medium / high
- **agent2_keyword_count** · matched suspicious words
- **agent2_phrase_count** · matched suspicious phrases

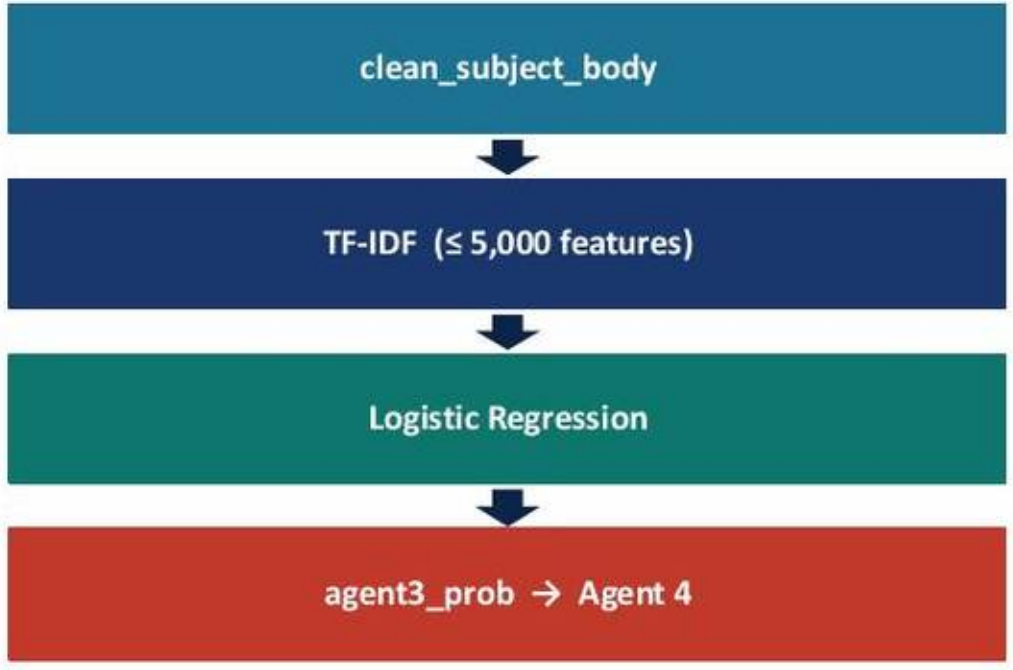
AGENT DESIGN

Agent 3 — NLP Classification

TF-IDF + Logistic Regression

Vectorizer	TfidfVectorizer (English stop-words removed)
Vocabulary	max_features = 10,000 tokens
Input field	clean_subject_body (from Agent 1)
Model	LogisticRegression(max_iter = 1000)
Datasets	Multi-source training + external testing
Outputs	Hard label, phishing probability per email

Pipeline Flow



- *show_top_features() exposes words with the largest positive / negative coefficients — useful for inspecting the learned vocabulary.*
- *Naïve Bayes was also evaluated for comparison, but Logistic Regression showed better overall performance*

Agent 3 — Model Selection Process

Model	Accuracy	Precision	Recall	F1-score
Agent3 Logistic Regression	0.9232707775	0.8795211739	0.9324508734	0.9052129562
Agent3 Naive Bayes	0.8374262735	0.928742515	0.6349617904	0.7542551467

AGENT DESIGN

Agent 4 — Reasoning & Decision

Combines outputs of all earlier agents through a small, interpretable Logistic Regression — issues the final binary verdict.

Feature Vector (7 columns)

`agent2_score`	weighted rule score
`agent2_risk_num`	low / med / high → 0 / 1 / 2
`agent2_keyword_count`	# matched keywords
`agent2_phrase_count`	# matched phrases
`has_url`	binary URL flag
`url_count`	# URLs in raw text
`agent3_prob`	TF-IDF / LR phishing probability

Design Choices

Hand-engineered features, not raw text

Forces the final decision through interpretable signals — readable coefficients, no opaque embeddings at the reasoning stage.

Learned weighting of rules + ML

agent3_prob enters as just another feature, so the rule engine and language model are combined through a learned weighting rather than a hand-crafted heuristic.

PIPELINE

End-to-End Data Flow

Each agent appends columns to a shared DataFrame; Agent 4 consumes the engineered feature vector.



Same multi-source training sets (random_state = 42) are reused by Agent 3 and Agent 4, while evaluation is performed on a separate external test set.

IMPLEMENTATION

Tools & Reproducibility

Stack

`Python 3`	Language runtime
`pandas`	DataFrame as inter-agent contract
`re`	Regex cleaning + phrase patterns
`NLTK`	word_tokenize · Porter stemming
`scikit-learn`	TfidfVectorizer · LogisticRegression · metrics
`matplotlib`	Confusion matrix and metric figures

Reproducibility Practices

Fixed seed	random_state = 42 across all splits
External evaluation	Separate external test dataset (w. 18,650 emails)
Shared training data	Agent 3 and Agent 4 use the same training datasets
Saved artifacts	metrics CSV + confusion matrix PNG
Single entry point	main.py orchestrates Agents 1 → 4

EXPERIMENTS

Evaluation Methodology

TRAIN	TEST	Threshold
56,896 emails	18,650 emails	0.8

Metrics Reported

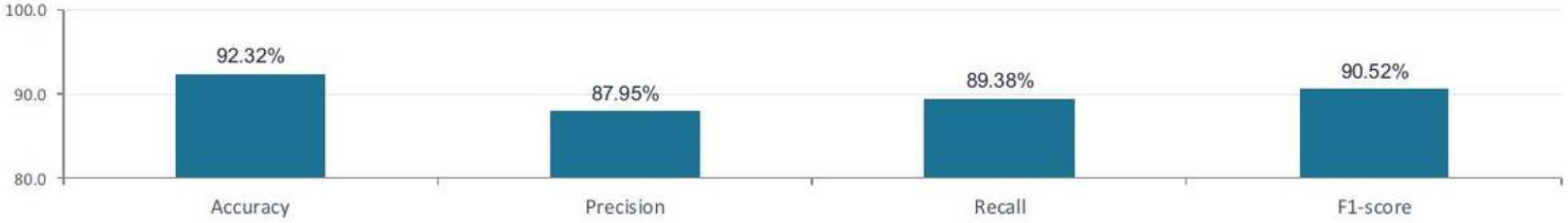
Accuracy	`(TP + TN) / (TP + TN + FP + FN)`	*Overall fraction classified correctly.*
Precision	`TP / (TP + FP)`	*Of mails flagged as phishing, how many really are?*
Recall	`TP / (TP + FN)`	*Of true phishing mails, how many did we catch?*
F1-score	`Harmonic mean of P and R`	*Balances false positives and false negatives.*
Confusion matrix	`TN, FP, FN, TP cells`	*Reveals the structure of residual errors.*

RESULTS

Headline Results — Agent 4

92.29%	90.85%	89.38%	0.9011
ACCURACY	PRECISION	RECALL	F1-SCORE

Side-by-side performance



RESULTS

Confusion Matrix (Test Set)

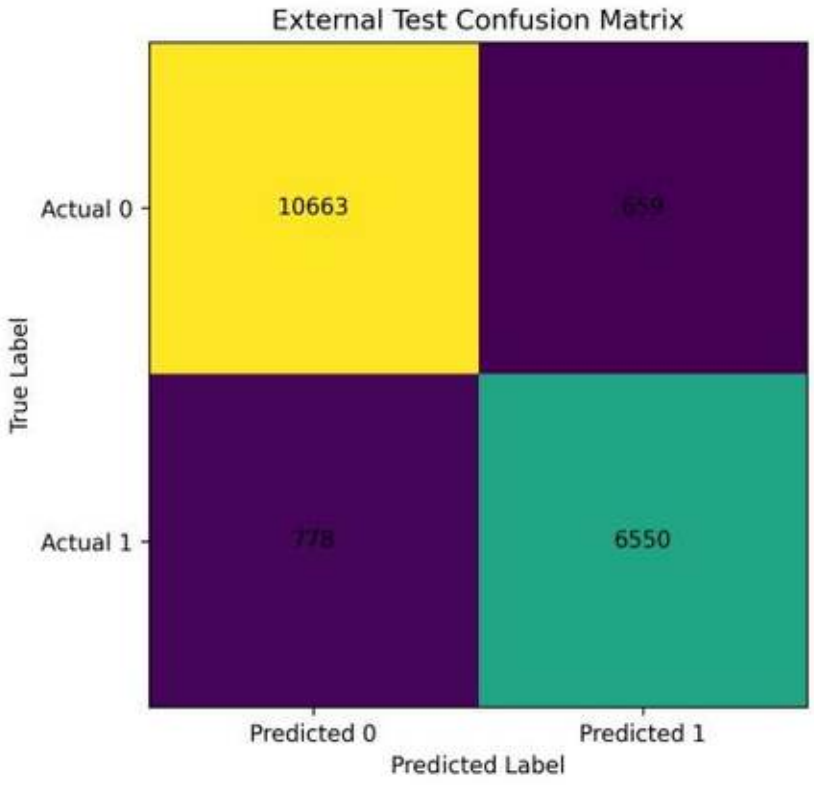


Fig. 1. Confusion matrix of Agent 4 on the External test set.

Cell counts

True Negatives (legit → legit)	**10,663**
False Positives (legit → phish)	**659**
False Negatives (phish → legit)	**778**
True Positives (phish → phish)	**6,550**

Only 778 phishing emails missed out of 10,663 — a 7.2% miss rate.
Critical in security: every undetected phish is a potential compromise.

RESULTS

Interpretability — Agent 4 Coefficients

Logistic Regression coefficients (Agent 4)



Bars scaled to coefficient magnitude · positive (blue) pushes toward phishing · negative (coral) pushes toward legitimate.

Takeaways

Text classifier dominates

agent3_prob carries a +6.127 weight — the TF-IDF / LR probability is the single strongest predictor.

Rule signals still matter

Keyword count, has_url, and risk_num all add positive evidence for phishing.

Calibration term

Negative weight on agent2_phrase_count suggests that Agent4 suppresses redundant phrase-based signals already captured by Agent 3.

DISCUSSION

Discussion — Wins and Challenges

WHAT WENT WELL

- Agent decomposition simplified development

Each stage validated in isolation — clean text, firing keywords, top tokens, readable coefficients.

- Strong end-to-end accuracy

92.29% on a realistic, public dataset — competitive with monolithic detectors.

- Interpretability preserved

Each Agent 4 coefficient maps to a human-readable feature.

CHALLENGES

- Messy CEAS_08 text

HTML fragments, embedded URLs, and encoding artifacts forced tighter Agent 1 cleaning rules.

- Subjective keyword weights

Per-token caps (3 / 2) and a 1.15× co-occurrence boost prevent any one signal from dominating.

- Avoiding data leakage

Agent 3 trains on a stratified split; Agent 4 reuses the same seed and split to avoid mixing train and test rows.

OUTLOOK

Limitations & Future Work

Current Limitations

Text-only signal

No headers, sender reputation, or attachment / link inspection.

Limited dataset diversity

Most datasets are English-only and may not fully represent evolving phishing styles.

Hand-curated rules

Agent 2 dictionaries and weights set manually.

Shallow text model

TF-IDF misses long-range linguistic cues a transformer would catch.

Next Steps

Upgrade Agent 3

Replace TF-IDF + LR with a transformer encoder (e.g., DistilBERT) for longer-range cues.

Auto-learn rules

Mine keyword and phrase weights for Agent 2 from labeled data.

Richer features

Add sender domain, reply-to consistency, attachment types, and per-URL characteristics (TLD, domain age).

Drift evaluation

Test on additional, more recent corpora and in an online setting.

CONCLUSION

An interpretable, modular, agent-based design that matches monolithic detectors.

92.29%	0.9085	0.8938	0.9011
ACCURACY	PRECISION	RECALL	F1-SCORE

Thank you. Questions?

Gabriel Olufade · Seongjae Shin · Imad Ali Syed | EE G7601 · Department of Cybersecurity